Michael Huang
EN.625.633.81
Homework 11

1.

We are given 40 annual counts, in thousands, of recruits R and spawners S in a salmon population. We are given The Beverton-Holt model for this relationship:

$$R = \frac{1}{\beta_1 + \beta_2/S}$$

To find the stable population level, we substitute $R = S$:

$$S = \frac{1}{\beta_1 + \beta_2/S}$$

$$S(\beta_1 + \beta_2/S) = 1$$

$$\beta_1 S + \beta_2 = 1$$

$$\beta_1 S = 1 - \beta_2$$

$$S = \frac{1 - \beta_2}{\beta_1}$$

(a)

We aim to use the bootstrap to obtain the 95% CI interval and standard error / estimated variance for the estimate, from two methods:
- Bootstrapping the residuals, i.e. resample only the residuals
- Bootstrapping the cases, i.e. resample entire pairs of (S, R) values

To bootstrap, we must linearize the model, since it's currently not in a linear form:

$$R = \frac{1}{\beta_1 + \beta_2/S}$$

$$\frac{1}{R} = \beta_1 + \beta_2 S = \beta_1 + \beta_2 \cdot \frac{1}{S}$$

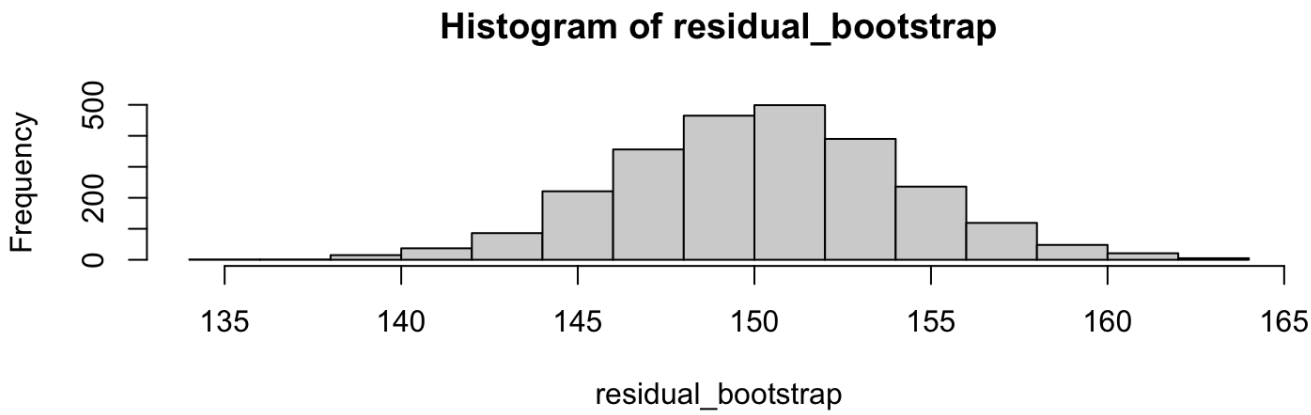
which is in linear form now, so we take the bootstrap based on this regression that has resulted from transforming the variables into $\frac{1}{R}$ and $\frac{1}{S}$.

```
> B = 2500
>
> R = c(68, 77, 299, 220, 142, 287, 276, 115, 64, 206, 222, 205, 233, 228,
```

```
+       188, 132, 285, 188, 224, 121, 311, 166, 248, 161, 226, 67, 201,
267,
+       121, 301, 244, 222, 195, 203, 210, 275, 286, 275, 304, 214)
>
> S = c(56, 62, 445, 279, 138, 428, 319, 102, 51, 289, 351, 282, 310,
+       266, 256, 144, 447, 186, 389, 113, 412, 176, 313, 162, 368, 54,
+       214, 429, 115, 407, 265, 301, 234, 229, 270, 478, 419, 490, 430,
235)
>
> R_inverse = 1 / R
> S_inverse = 1 / S
>
> salmon_regression <- lm(R_inverse ~ S_inverse)
> # intercept
> beta1_hat = coef(salmon_regression)[1]
> # slope
> beta2_hat = coef(salmon_regression)[2]
>
> S_stable <- (1 - beta2_hat) / beta1_hat
>
> residual_bootstrap = rep(0, B)
> case_bootstrap = rep(0, B)
>
> original_residuals = residuals(salmon_regression)
> original_fitted = fitted(salmon_regression)
>
> for (j in 1:B) {
+   n = length(R)
+   sample_index = sample(1:n, n, replace = TRUE)
+
```

```
+   # Bootstrap the residuals
+   resampled_residuals = salmon_regression$residuals[sample_index]
+   altered_response = resampled_residuals + predict(salmon_regression)
+
+   new_regression = lm(altered_response ~ S_inverse)
+   new_beta1 = new_regression$coefficients[1]
+   new_beta2 = new_regression$coefficients[2]
+
+   residual_bootstrap[j] = (1 - new_beta2) / new_beta1
+
+   # Bootstrap the cases
+   resampled_R = R[sample_index]
+   resampled_S = S[sample_index]
+   resampled_R_inverse = 1 / resampled_R
+   resampled_S_inverse = 1 / resampled_S
+
+   new_regression = lm(resampled_R_inverse ~ resampled_S_inverse)
+   new_beta1 = new_regression$coefficients[1]
+   new_beta2 = new_regression$coefficients[2]
+
+   case_bootstrap[j] = (1 - new_beta2) / new_beta1
+ }
>
> # 95% Confidence Interval
> residual_ci = quantile(residual_bootstrap, c(0.025, 0.975))
> case_ci = quantile(case_bootstrap,  c(0.025, 0.975))
>
> # Standard Error / Estimated Variance
> residual_se = sd(residual_bootstrap);
> case_se = sd(case_bootstrap);
```

```
>
> print("Residuals Results")
[1] "Residuals Results"
> sprintf("95%% CI: [%f, %f]", residual_ci[1], residual_ci[2])
[1] "95% CI: [142.450819, 158.361046]"
> print(paste("Standard Error:", residual_se))
[1] "Standard Error: 4.02236150381366"
>
> print("Case Results")
[1] "Case Results"
> sprintf("95%% CI: [%f, %f]", case_ci[1], case_ci[2])
[1] "95% CI: [142.609496, 157.933674]"
> print(paste("Standard Error:", case_se))
[1] "Standard Error: 3.84970115387248"
>
> # Histograms
> par(mfrow = c(2,1))
> hist(residual_bootstrap)
> hist(case_bootstrap)
```



We see that the two distributions are similar in shape and both center around 150,000, with the standard errors also being fairly close to around 4,000 fish and with similar bounds for the confidence intervals. We note that the residual bootstrap looks nicer with having a much larger distribution of results and having a more pleasing, normal distribution-like shape when compared to the case/pair bootstrap results. However, we know that the difference between these methods is that in the residual bootstrap, we completely believe the proposed regression model and believe that the variance in results is independently distributed and doesn't depend on S at all, keeping it constant; this in turn leads to the pretty and normalized results we see. Bootstrapping the cases/pairs lets S and R vary together and captures the data structure more purely than trying to fit an exact model, so it has a better fit for the randomness and greater variance of the actual situation, which in turn leads to a much more spikey, centralized distribution of results.

(b)

We calculate the bias in the estimates accordingly:

```
> # Bias
> residual_bias = mean(residual_bootstrap) - S_stable
> case_bias = mean(case_bootstrap) - S_stable
>
> print(paste("Original S Esimate:", S_stable))
[1] "Original S Esimate: 150.097634300979"
> print(paste("Residual bootstrap bias:", residual_bias))
[1] "Residual bootstrap bias: 0.0659949321566273"
> print(paste("Case bootstrap bias:", case_bias))
[1] "Case bootstrap bias: 0.0359000321315364"
```

While there is more bias for the residual bootstrap method, the bias is extremely small relative to the actual estimate and standard errors that we found. Empirically, it seems that there is very little bias in the estimate, if any, and both bootstrap methods indicating very small bias also reinforces this conclusion.

2.

We generate 100 independent samples, each of size $n = 20$, from $N(\theta, 1)$ with $\theta = 1$.

(a)

For each sample, we need to compute the bootstrap and jackknife estimates for the variance $\hat{\theta} = \bar{X}$, and then compute the mean and standard deviation of these variance estimates over the 100 samples.

```
> num_samples = 100
> n = 20
> B = 2500
> theta = 1
```

```
>
> bootstrap_variances_x = rep(0, num_samples)
> jackknife_variances_x = rep(0, num_samples)
>
> for (i in 1:num_samples) {
+
+   x = rnorm(n, mean = theta, sd = 1)
+   theta_hat = mean(x)
+
+   bootstrap_estimates = rep(0, B)
+   for (j in 1:B) {
+     bootstrap_sample = sample(x, n, replace = TRUE)
+     bootstrap_estimates[j] = mean(bootstrap_sample)
+   }
+   bootstrap_variances_x[i] = var(bootstrap_estimates)
+
+   jackknife_estimates = rep(0, n)
+   for (k in 1:n) {
+     jackknife_estimates[k] = mean(x[-k])
+   }
+   jackknife_variances_x[i] = (
+     (n - 1) / n) *
+     sum(
+       (jackknife_estimates - mean(jackknife_estimates)
+     )^2
+   )
+ }
>
> print(paste("Real variance of X:", 1/n))
[1] "Real variance of X: 0.05"
```

```
> print(paste("Bootstrap Mean for Variance of X:", mean(bootstrap_variances_x
[1] "Bootstrap Mean for Variance of X: 0.0478879366818966"
> print(paste("Bootstrap SD for Variance of :", sd(bootstrap_variances_x)))
[1] "Bootstrap SD for Variance of X: 0.0156345979679424"
> print(paste("Jackknife Mean for Variance of X:", mean(jackknife_variances_x
[1] "Jackknife Mean for Variance of X: 0.0503804853588733"
> print(paste("Jackknife SD for Variance of X:", sd(jackknife_variances_x)))
[1] "Jackknife SD for Variance of X: 0.0163058104620382"
```

(b)

We do the same for $\hat{\theta} = \bar{X}^2$:

```
> bootstrap_variances_x2 = rep(0, num_samples)
> jackknife_variances_x2 = rep(0, num_samples)
>
> for (i in 1:num_samples) {
+
+   x = rnorm(n, mean = theta, sd = 1)
+   theta_hat = mean(x)^2
+
+   bootstrap_estimates = rep(0, B)
+   for (j in 1:B) {
+     bootstrap_sample = sample(x, n, replace = TRUE)
+     bootstrap_estimates[j] = mean(bootstrap_sample)^2
+   }
+   bootstrap_variances_x2[i] = var(bootstrap_estimates)
+
+   jackknife_estimates = rep(0, n)
+   for (k in 1:n) {
+     jackknife_estimates[k] = mean(x[-k])^2
+   }
```

```
+   jackknife_variances_x2[i] = (
+     (n - 1) / n) *
+     sum(
+       (jackknife_estimates - mean(jackknife_estimates)
+       )^2
+     )
+ }
>
> print(paste("Real variance of X^2:", ((4 * n) + 2) / n^2))
[1] "Real variance of X^2: 0.205"
> print(paste("Bootstrap Mean for Variance of X^2:", mean(bootstrap_variance
[1] "Bootstrap Mean for Variance of X^2: 0.223126002866947"
> print(paste("Bootstrap SD for Variance of X^2:", sd(bootstrap_variances_x2
[1] "Bootstrap SD for Variance of X^2: 0.121789993657577"
> print(paste("Jackknife Mean for Variance of X^2:", mean(jackknife_variance
[1] "Jackknife Mean for Variance of X^2: 0.229059258981623"
> print(paste("Jackknife SD for Variance of X^2:", sd(jackknife_variances_x2
[1] "Jackknife SD for Variance of X^2: 0.128986706299355"
```

Both of the methods are pretty accurate for $\hat{\theta} = \bar{X}$, with the mean for the variance for both the bootstrap and jackknife methods being very close to the actual variance value, and the standard deviations also being almost equal and still very low. For $\hat{\theta} = \bar{X}^2$, however, the means for the variance of the value become a little more off, with the jackknife being a bit worse than the bootstrap. We also see that the standard deviations are much higher than with $\bar{X}$. We can infer this as being since $\bar{X}^2$ is nonlinear, so the estimates both become worse overall. The jackknife in particular likely does worse than the bootstrap because of the leave-one-out methodology's struggle with the quickly changing and less smooth statistics.

3.

We aim to find the number of bootstraps necessary to accurately calculate a 95% confidence interval in the difference of two population means.

The plan is to simulate n values of $X_1 \sim N(\mu_1, \sigma_1^2)$ and m values of $Y \sim N(\mu_2, \sigma^2)$ and calculate a 95% confidence interval in $\mu_1 - \mu_2$. Then, calculate a 95% confidence interval for $\mu_1 - \mu_2$ using the bootstrap techniques learned in the module, that is:
- Sample bootstramp sample X^* with replacement
- Sample bootstrap sample Y^* with replacement
- Calculate $\mu_1 - \mu_2$ for each bootstrap sample
- Take the 95% confidence interval in the middle as normal

```
> B_values = c(100, 500, 1000, 2500, 5000, 10000)
> # Repeat multiple times to get to steady state
> repetitions = 100
>
> n = 50
> mu_1 = 3
> sigma_1 = 1
>
> m = 50
> mu_2 = 4
> sigma_2 = 2
>
> X = rnorm(n, mean = mu_1, sd = sigma_1)
> Y = rnorm(m, mean = mu_2, sd = sigma_2)
>
> # For calculated CI, use Welch's t-test which does not assume equal varian
> normal_t_test = t.test(X, Y, conf.level = 0.95)
```

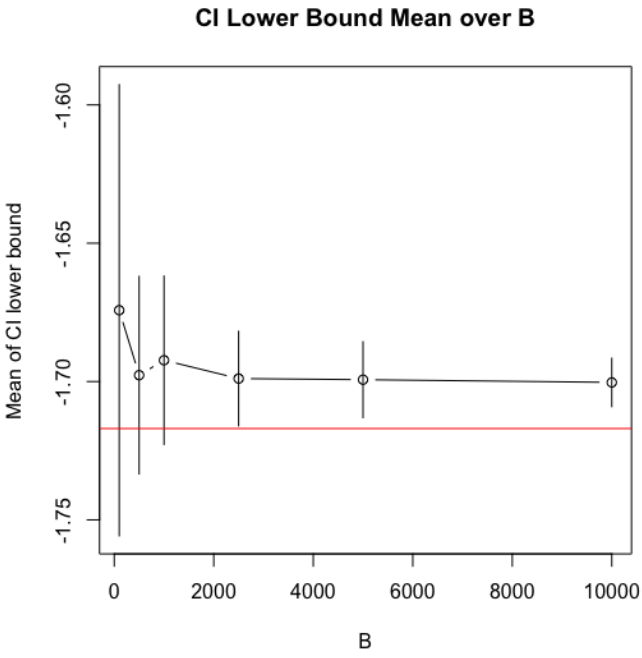
```
> calculated_ci = normal_t_test$conf.int
>
> bootstrap_ci = function(X, Y, B) {
+   bootstrap_differences = rep(0, B)
+   for (j in 1:B) {
+     boot_x = sample(X, n, replace = TRUE)
+     boot_y = sample(Y, m, replace = TRUE)
+
+     bootstrap_differences[j] = mean(boot_x) - mean(boot_y)
+   }
+
+   bootstrap_ci = quantile(bootstrap_differences, probs = c(0.025, 0.975))
+
+ }
>
> bootstrap_results = data.frame(B = integer(), ci_lower = numeric(), ci_upp
= numeric())
> for (B in B_values) {
+
+   lower_results = rep(0, repetitions)
+   upper_results = rep(0, repetitions)
+   for (r in 1:repetitions) {
+     bootstrap_result = bootstrap_ci(X, Y, B)
+     lower_results[r] = bootstrap_result[1]
+     upper_results[r] = bootstrap_result[2]
+   }
+
+   bootstrap_results = rbind(bootstrap_results, data.frame(
+     B = B,
+     lower_mean = mean(lower_results),
```

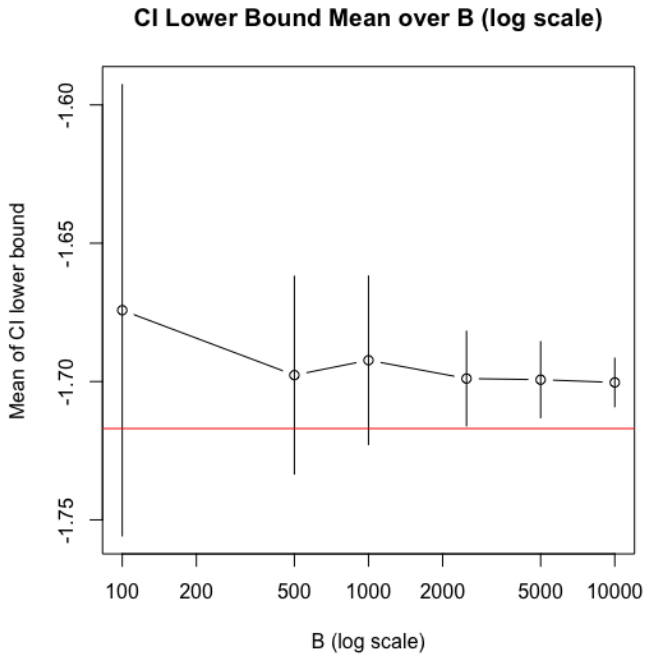
```
+     lower_sd = sd(lower_results),
+     upper_mean = mean(upper_results),
+     upper_sd = sd(upper_results)
+   ))
+ }
>
> sprintf("Calculated CI: [%f, %f]", calculated_ci[1], calculated_ci[2])
[1] "Calculated CI: [-1.716995, -0.326697]"
>
> print(bootstrap_results)
      B lower_mean    lower_sd upper_mean    upper_sd
1   100  -1.674224 0.081550883 -0.3683828 0.075172619
2   500  -1.697692 0.035720486 -0.3540524 0.041719129
3  1000  -1.692301 0.030432004 -0.3520838 0.028434478
4  2500  -1.698923 0.017086303 -0.3496306 0.018933560
5  5000  -1.699331 0.013708461 -0.3500792 0.012151156
6 10000  -1.700329 0.008748158 -0.3493192 0.008897934
>
> # Lower means, normal and log
> par(mfrow = c(1, 2))
> plot(bootstrap_results$B, bootstrap_results$lower_mean,
+      type = "b",
+      ylim = range(c(bootstrap_results$lower_mean - bootstrap_results$lower_
+                     bootstrap_results$lower_mean + bootstrap_results$lower_
+      xlab = "B",
+      ylab = "Mean of CI lower bound",
+      main = "CI Lower Bound Mean over B")
> arrows(bootstrap_results$B,
+        bootstrap_results$lower_mean - bootstrap_results$lower_sd,
+        bootstrap_results$B,
```

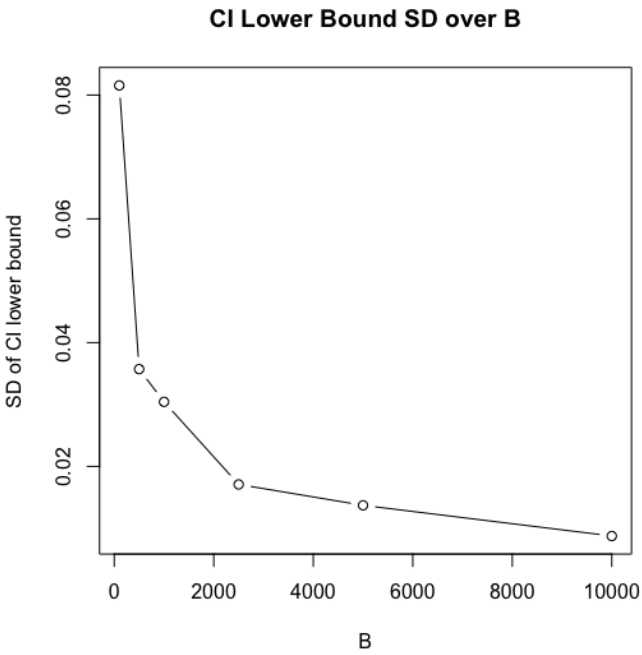
```
+        bootstrap_results$lower_mean + bootstrap_results$lower_sd,
+        code=0
+ )
> abline(h = calculated_ci[1], col = "red")
> plot(bootstrap_results$B, bootstrap_results$lower_mean,
+      type = "b",
+      log = "x",
+      ylim = range(c(bootstrap_results$lower_mean - bootstrap_results$lower_
+                     bootstrap_results$lower_mean + bootstrap_results$lower_
+      xlab = "B (log scale)",
+      ylab = "Mean of CI lower bound",
+      main = "CI Lower Bound Mean over B (log scale)")
> arrows(bootstrap_results$B,
+        bootstrap_results$lower_mean - bootstrap_results$lower_sd,
+        bootstrap_results$B,
+        bootstrap_results$lower_mean + bootstrap_results$lower_sd,
+        code=0
+ )
> abline(h = calculated_ci[1], col = "red")
>
> # Lower SDs, normal and log
> par(mfrow = c(1, 2))
> plot(bootstrap_results$B, bootstrap_results$lower_sd,
+      type = "b",
+      xlab = "B",
+      ylab = "SD of CI lower bound",
+      main = "CI Lower Bound SD over B")
> plot(bootstrap_results$B, bootstrap_results$lower_sd,
+      type = "b",
+      log = "x",
```

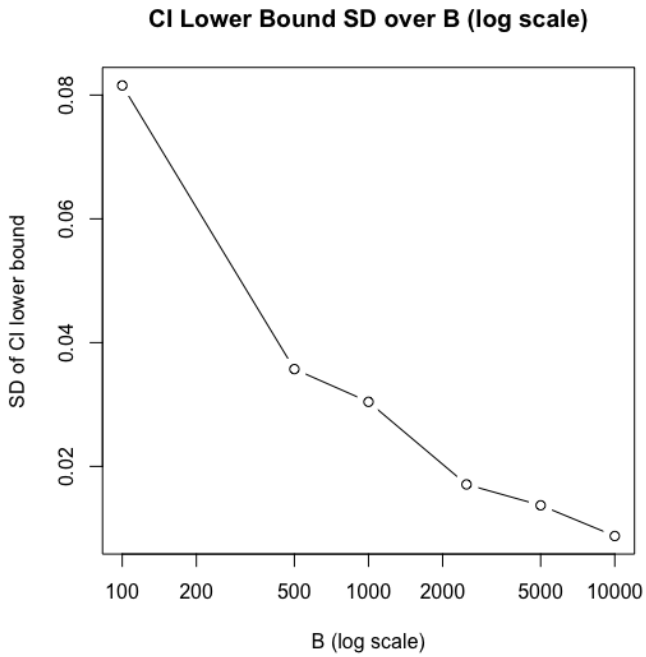
```
+      xlab = "B (log scale)",
+      ylab = "SD of CI lower bound",
+      main = "CI Lower Bound SD over B (log scale)")
>
> # Upper means, normal and log
> par(mfrow = c(1, 2))
> plot(bootstrap_results$B, bootstrap_results$upper_mean,
+      type = "b",
+      ylim = range(c(bootstrap_results$upper_mean - bootstrap_results$upper
+                     bootstrap_results$upper_mean + bootstrap_results$upper
+      xlab = "B",
+      ylab = "Mean of CI Upper Bound",
+      main = "CI Upper Bound Mean over B")
> arrows(bootstrap_results$B,
+        bootstrap_results$upper_mean - bootstrap_results$upper_sd,
+        bootstrap_results$B,
+        bootstrap_results$upper_mean + bootstrap_results$upper_sd,
+        code=0
+ )
> abline(h = calculated_ci[2], col = "red")
> plot(bootstrap_results$B, bootstrap_results$upper_mean,
+      type = "b",
+      log = "x",
+      ylim = range(c(bootstrap_results$upper_mean - bootstrap_results$upper
+                     bootstrap_results$upper_mean + bootstrap_results$upper
+      xlab = "B (log scale)",
+      ylab = "Mean of CI Upper Bound",
+      main = "CI Upper Bound Mean over B (log scale)")
> arrows(bootstrap_results$B,
+        bootstrap_results$upper_mean - bootstrap_results$upper_sd,
```

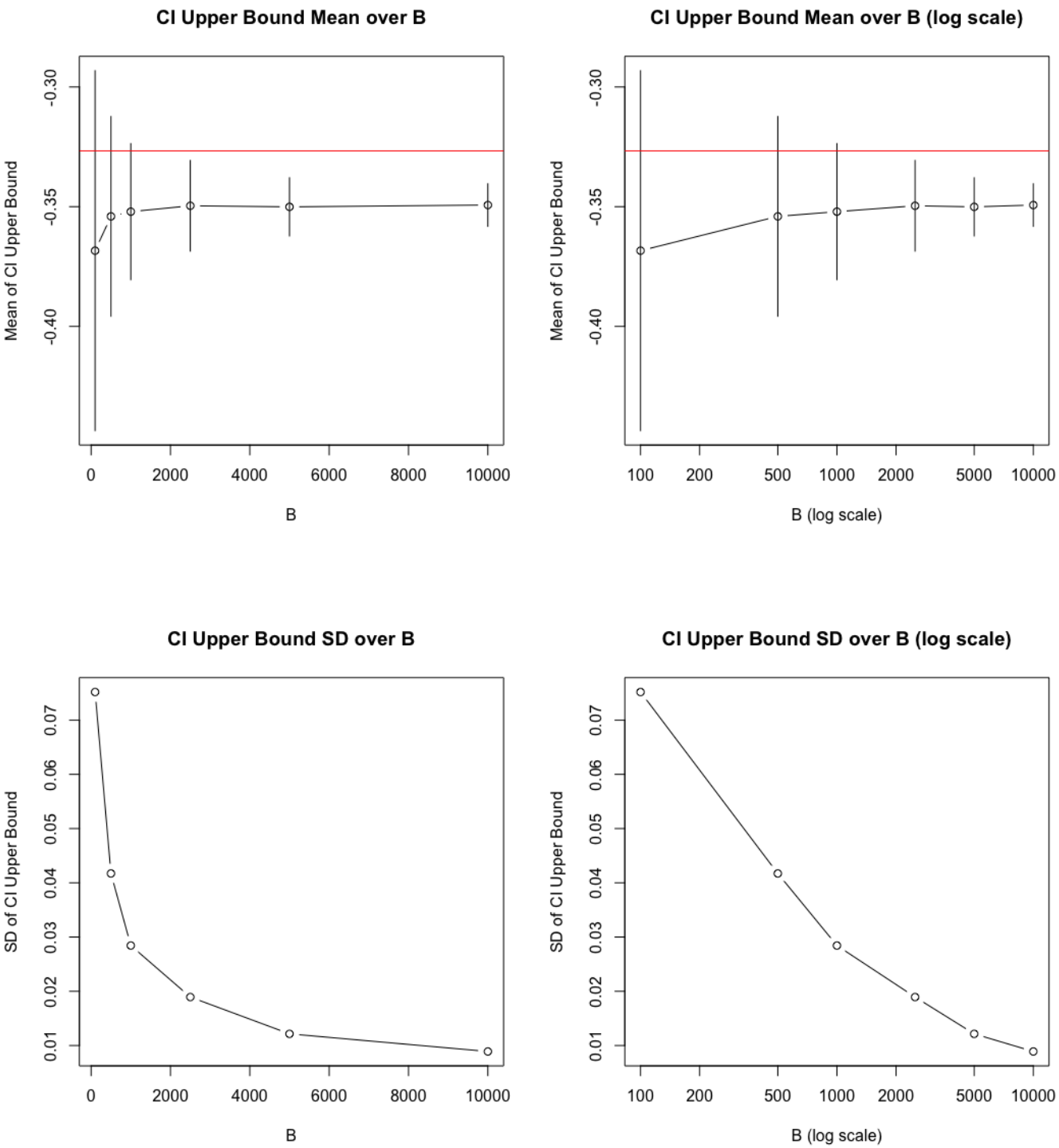
```
+        bootstrap_results$B,
+        bootstrap_results$upper_mean + bootstrap_results$upper_sd,
+        code=0
+ )
> abline(h = calculated_ci[2], col = "red")
>
> # Upper SDs, normal and log
> par(mfrow = c(1, 2))
> plot(bootstrap_results$B, bootstrap_results$upper_sd,
+      type = "b",
+      xlab = "B",
+      ylab = "SD of CI Upper Bound",
+      main = "CI Upper Bound SD over B")
> plot(bootstrap_results$B, bootstrap_results$upper_sd,
+      type = "b",
+      log = "x",
+      xlab = "B (log scale)",
+      ylab = "SD of CI Upper Bound",
+      main = "CI Upper Bound SD over B (log scale)")
```


CI Lower Bound Mean over B
Mean of CI lower bound
-1.60
-1.65
-1.70
-1.75
0
2000
4000
6000
8000
10000
B


CI Lower Bound Mean over B (log scale)
Mean of CI lower bound
-1.60
-1.65
-1.70
-1.75
100
200
500
1000
2000
5000
10000
B (log scale)


CI Lower Bound SD over B
SD of CI lower bound
0.08
0.06
0.04
0.02
0
2000
4000
6000
8000
10000
B


CI Lower Bound SD over B (log scale)
SD of CI lower bound
0.08
0.06
0.04
0.02
100
200
500
1000
2000
5000
10000
B (log scale)



Looking at the figures and results, we see that there are generally diminishing returns in terms of getting meaningfully reduced range of mean differences past the $B = 2500$ point. The SD does continue to improve, however. Clearly, $B = 100$ is not enough, $B = 500$ is a decent improvement, and then it begins to become better but in diminishing amounts as we approach $B = 2500$. This makes sense,

given how the bootstrap works relative to a theoretically calculated amount. In my opinion, the threshold seems good around there, however, with the red lines of the calculated bounds, it is notable that the bootstrap will never reach the point of exactly becoming dependably exactly as accurate as the theoretical mean bounds. This makes sense, given the point of using bootstrap and the tradeoff here, so my B value evaluation criteria was more geared toward hitting a point of relative accuracy is getting as close as we can seemingly get very dependably.